1.

显然 (众所周知) 地只需要判断 G 中是否存在奇环, 存在 $\mathcal{O}(|V|+|E|)$ 的搜索算法.

2.

设 NDTM N_1 识别 L_1, N_2 识别 L_2.

$L_1 \cup L_2 \in$ NP. 构造 N, 从 q_0 直接非确定性转移到 $q_0^{N_1}$ 或者 $q_0^{N_2}$ 即可.

$L_1 \cap L_2 \in$ NP. 构造 N, $q_0 = q_0^{N_1}$, 转移同 N_1, 并将 $q_{\text{accept}}^{N_1}$ 替换为 $q_0^{N_2}$, 此后转移同 N_2.

3.

不妨设两个图结点数均为 n. 要求证书编码映射 $f: V_G \to V_H$. 由于 $|\{\text{isom. } f: V_G \to V_H\}| = n!$, 编码能够在 $\log n! = \mathcal{O}(n \log n) = o(n^2)$ 的多项式长度完成. 验证机根据映射, 逐个检查边的一致性即可.

4.

(1) 若两个 literal 真值不同, 它们在取反的赋值里真值也不同.

(2) 一方面, 如果一个 $3\mathrm{SAT}$ 可满足, 对每个字句 $c_i = y_1 \vee y_2 \vee y_3$, 设它如题所属映射到两个字句, 讨论 $\neq\mathrm{SAT}$ 中的赋值:

- 若 $y_1 \neq y_2$ 或者 $y_1 = y_2 = \mathrm{True}$, 则取 $z_i = \mathrm{False}$.
- 否则 $y_1 = y_2 = \mathrm{False}$, 则取 $z_i = \mathrm{True}$, 此时必然 $y_3 = \mathrm{True}$.
- 总是取 $b = \mathrm{False}$.

已然满足. 另一方面, 若这样一个 $\neq\mathrm{SAT}$ 可满足, 根据 (1), 我们总能适当反转赋值使 $b = \mathrm{False}$, 接着讨论:

- 若 $y_1 = y_2 = \mathrm{False}$, 则 $z_i = \mathrm{True}$, $\overline{z_i} = \mathrm{False}$, 而 $b = \mathrm{False}$, 所以 $y_3 = \mathrm{True}$;
- 否则 $y_1 \vee y_2 = \mathrm{True}$.

因而已然给出一个合法的 $3\mathrm{SAT}$ 赋值.

(3) 综上, $3\mathrm{SAT} \leq_{\mathrm{p}} \neq\mathrm{SAT}$, 前者是 NP-Complete 的, 所以后者也是 NP-Complete 的.

5.

只需证明 $A \in$ P. 由于 A 是 NP-Complete 的, 存在多项式可计算的 $f: \mathrm{CNF} \to \mathbb{N}$ 满足 $\varphi \in \mathrm{SAT} \Leftrightarrow 1^{f(\varphi)} \in A$, 注意这自然给出 $f(\varphi) = \mathcal{O}(\mathrm{poly}(|\varphi|))$, 且 $f(\varphi) = f(\psi) \Rightarrow (\varphi \in \mathrm{SAT} \leftrightarrow \psi \in \mathrm{SAT})$. 以此构造描述如下算法的 TM:

```
M ← Map_{ℕ→Bool} {}
function solve(φ) → Bool :
  if φ has no variable then
    return Bool(φ)
  k ← f(φ)
  if k ∈ M then
    return M[k]
  x ← head(varlist(φ))
  M[k] ← solve(φ[x = False]) ∨ solve(φ[x = True])
  return M[k]
```

初始调用为 $\mathrm{solve}(\varphi)$, $|\varphi| = n$, 函数遍历到的 CNF 集合为 Φ, 则 $\forall \varphi \in \Phi,\ |\varphi| \le n$. 因而

$$\exists m = \mathrm{poly}(n),\ \forall \varphi \in \Phi,\ f(\varphi) \le m.$$

所以, solve 只可能遍历到 $\mathcal{O}(m) = \mathcal{O}(\mathrm{poly}(n))$ 个状态, 每层递归内部复杂度为 $\mathcal{O}(n)$. 而根据 f 的上述性质, solve 的记忆化策略正确, 因而搜索结果正确判定 φ 是否可满足. 最终, solve 在 $\mathcal{O}(n\,\mathrm{poly}(n)) = \mathcal{O}(\mathrm{poly}(n))$ 的复杂度内解决了 SAT, $\mathrm{SAT} \in \mathsf{P}$. 而 SAT 是 NP-Complete 的, 我们得到 $\mathsf{P} = \mathsf{NP}$.

6.

已证明 $\mathrm{INDSET} \in \mathsf{NP}$, 若 $\mathsf{P} = \mathsf{NP}$, 则 $\mathrm{INDSET} \in \mathsf{P}$, 那么存在 TM $M(G, k)$ 在多项式时间判定 G 中是否存在大小为 k 的独立集. 以此构造描述如下算法的 TM (设 $G = (V, E)$, $G \setminus S$ 表示 G 去掉点集 S 中的点及其连边得到的图):

```
1  k ← n
2  while M(G, k) = R do
3    k ← k − 1
4  S ← {}
5  while k > 0 do
6    for u ∈ V do
7      G′ ← G \ {v : u = v ∨ (u, v) ∈ E}
8      if M(G′, k − 1) = A then
9        (S, k, G) ← (S ⊔ {u}, k − 1, G′)
10       break
11 output S
```

在第 8 行, 由于 $M(G, k) = \mathrm{A}$ 归纳地总是成立, 当 u 取到证书中任意结点 (也即大小为 k 的独立集中任意结点) 时, 我们会将它加入当前独立集 S, 并将 u 和 u 的所有邻接点删去. 最终 S 一定是大小为 k 得到独立集, 也即最大独立集.

复杂度为 $\mathcal{O}(n^2 f(n))$, 其中 $f(n)$ 为 M 的复杂度. 这是多项式算法.

7.

反证. 设单带 TM $M \in \mathrm{DTIME}(f(n))$, $f(n) = o(n \log n)$ 而 M 识别的 A 不正则. 记 $\mathrm{cs}_x(i)$ 表示将 x 作为输入时, M 的读写头穿过纸带 i 和 $i+1$ 边界时的机器状态按照运行时间升序排列得到的状态序列, 即交叉序列, $|\mathrm{cs}_x(i)|$ 表示其长度.

先证引理: 若 $x = uvw$ 满足 $|u| = p$, $|uv| = q$, 且 $\mathrm{cs}_x(p) = \mathrm{cs}_x(q)$, 则 $M(x) = M(uw)$. 这是因为, $|\mathrm{cs}_x(p)| = |\mathrm{cs}_x(q)|$ 给出 M 在停机 (或者陷入死循环) 时一定不处于 v 内部, 那么 M 在 v 内部的运行行为完全由离开 v 的状态决定, 即由 v 两端的交叉序列. 而 v 两端交叉序列与 ε 相同, 所以 $M(x) = M(uvw) = M(u\varepsilon w) = M(uw)$.

由于 A 不正则, 根据 Myhill-Nerode 定理, A 中存在无穷多个前缀等价类. 因此, 对任意大的 $n \in \mathbb{N}$, 总存在一个 x, 它是其所属等价类中最短的代表元且满足 $|x| \geq n$. 根据引理, 若 x 内部存在两个位置产生了相同的交叉序列, 则这两个位置之间的串可以删去得到 x', 且 $x' \equiv x$, 与 x 的最短性矛盾. 所以 x 内部的交叉序列两两不同. 据此, 考虑 $M(x)$ 的运行时间

$$T(n) \geq \sum_{i=1}^{n} \mathrm{cs}_x(i) \geq \min_{\{c_j\}} \left\{ \sum_{j=0}^{\infty} j \cdot c_j \mid \sum_{j=0}^{\infty} c_j = n \wedge \left(\forall j,\ c_j \leq |Q|^j\right) \right\}.$$

不妨设 $|Q| = s$, $n = s^{k+1} - 1$, 则

$$T(n) \geq \sum_{j=0}^{k} j s^j \geq k s^k = \Theta(kn) = \Theta(n \log n).$$

这与 $M \in \mathrm{DTIME}(f(n))$ 矛盾. 因而 A 必须正则, 这样的 M 只能识别正则语言.